

FRED TALKS

4th September 2026

CONTENTS

Building agents that reach production systems with MCP

CLAUDE.COM · 1746 WORDS

Headless everything for personal AI

INTERCONNECTED · 1335 WORDS

The Reverse Information Paradox

SN SCRATCHPAD · 819 WORDS

Surrender as a non-stupid life strategy

SASHA CHAPIN · 2104 WORDS

Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · [SOURCE](#)

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the $M \times N$ integration problem.

Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it's thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

subsequently met that person. She was completely lovely—I find much to criticize in my own behavior from that period and nothing bad to say about her. And yet the relationship did not stick. How odd.

5.2 When I insist that a relationship must survive, I begin to resent it—I become burdened by all of the little moments of inauthenticity engaged in to shield the other person. The person becomes an obstacle, someone to navigate around. When I accept that any relationship could end, and surrender to the reality of people finding their own level, I become free to relate openly, and then actual closeness happens.

5.3 Panic is a part of love. I’ve learned that as I enter into any relationship that will alter me for the better, the manager in my head will come up with convincing reasons why it’s all wrong. This, too, is the refusal of the call; it is the fear of personal transformation masquerading as reason.

5.4 The older I get, the more that friendship is a discovered property. There are lots of wonderful people I meet who aren’t my friends. And then I meet someone and think: oh, that’s my friend, you’re coming with me. One of the best things you can do for yourself is to notice when you’re inexplicably bound to someone, and act on it without thinking.

5.5 We are drawn towards people because they are different. We are given the opportunity to love more when we find ourselves resenting them for the very difference that drew us to them. The opportunity is to release resentment and instead surrender to separateness. Ironically, this is how you feel less alone.

Thank you to Erin Shetron for editorial assistance. This essay was inspired a little bit by The Surrender Experiment, and a lot by [this totally wonderful meditation guide](#). Photo credit goes to Daido Moriyama.

1 [find in text]

This format is shamelessly stolen from [this fascinating essay](#) by “de Pony Sum.”

2 [find in text]

I first heard this point made by Adyashanti.

...

Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We’re already seeing this in adoption. The [MCP SDKs](#) recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including [Claude Cowork](#), [Claude Managed Agents](#), and [channels in Claude Code](#).

As MCP continues to support production agentic systems, we’re sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

Building effective MCP servers

We have over 200 MCP servers in our [directory](#), used by millions of people every day. From working closely with enterprises and developers building on the protocol, we’ve spotted a handful of design patterns that determine how reliably agents can use a server.

Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single `create_issue_from_thread` tool beats `get_thread` + `parse_messages` + `create_issue` + `link_attachment`. See [writing effective tools for agents](#) to learn more about the full pattern.

Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. [Cloudflare's MCP server](#) is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

Ship rich semantics where they help

[MCP Apps](#) is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

[Elicitation](#) lets your server pause mid-tool call to ask the user for input. [Form mode](#) sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. [URL mode](#) hands the user to a browser—use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest [MCP spec](#) supports [CIMD](#) (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Happiness

4.1 The norm for human life: gross injustice, corrupt rulers, disease and death, unequal fortunes, looming apocalypses. It's possible that a large portion of our species will die off—it's happened before, we've almost gone extinct previously, that's allowed in the game. If your basic needs are met but you cannot be content with the way it is now, that is not the fault of the way things are now. It's because you haven't surrendered to the basic properties of incarnating in human form.

4.2 Strangely, happiness isn't produced by getting what you want. Unhappiness is the product of the number of times per minute that you believe circumstances should be otherwise. “Enough” and “not enough” are mental stances that, very often, are uncorrelated with exterior facts. Perhaps this is one reason Jesus says it is “easier for a camel to go through the eye of a needle than for a rich man to enter the kingdom of God”—the more you focus on expanding your pile, the more ravenousness becomes a habit. By contrast, you can, right now, try believing that what's happening is what's supposed to be happening, in exactly the right order.

4.3 Satisfaction comes from giving yourself away. From realizing that you have been called to some occasion that requires everything from you, and saying, fine, fuck it, here I am, tear me apart, I will exhaust myself to set right this one irrelevant corner. It is possible to view your whole existence this way, as a process of giving away all, everything, of what was never yours.

4.4 Another profound source of satisfaction is fit: when you fit like a slender hand in the glove of your circumstances. Consider the possibility that this satisfaction is already trying to happen, but you are resisting it, in a doomed attempt to be happy. Settling into this fit is another way of describing what I mean by “doing what God seems to want.”

4.5 I can't defend this intellectually, but it has been true in my experience: spontaneous compassion is the approach to ethics that feels best and works most often. If you want a plan for consistently being good, the dumbest one that will work, and therefore the most robust, is “notice that prosocial behavior is nourishing, and that peace is only available if you're on good terms with your conscience, and allow behavior to flow from that.” I'm not deluded enough to think this is always sufficient, but it's a good start.

Love

5.1 I am capable of privately spoiling my wonderful marriage at any time, by noticing that it's not the relationship I would've imagined being in. Meanwhile, a few months before meeting my wife, I wrote down a list of characteristics that the perfect partner would have, and I

Goals, outcomes, effort

3.1 Discovered goals are entirely different from preconceived goals. Let's say you notice that when you play guitar and sing, people stop what they're doing to stare at you. The discovered goal of being a musician has presented itself. You are more likely to be successful than someone who privately dreams of being a musician without prompting from existence. Noticing such opportunities requires a lack of some previously decided agenda.

3.2 I've known many impressive people. Almost none of them started with a comprehensive vision of the grand work they would produce. Instead, they simply got into escalating amounts of trouble, and kept riding the trouble wave as far as it would go.

3.3 Excellence requires non-attachment to principles. All mediocre performers are the same: instead of improving on feedback from what happens, they react to what they think is supposed to be happening, or try to reinforce a theory they've internalized—perhaps an inaccurate approximation of someone they admire. This attempt at coherence is futile, since the mind of an expert makes no sense. It is filled with exceptions, contradictory principles butting up against each other. Perhaps the knowledge of how your life works is the same.

3.4 When a project in my life is working, it's typically marked by a sense of leverage. It's as if I've fallen into an updraft and it's propelling me. Meanwhile, when I'm privately excited about something, but there's no cooperation from the world, nothing comes together. The sensible conclusion is that my enthusiasm, alone, is not a helpful measurement.

3.5 It's nice to enjoy what you're doing. But there is a greater liberation available: accepting that the Work in front of you is your Work, and that has little to do with how you feel.

3.6 Another property of things that work out for me: they don't make sense to my rationalizing mind. There is the thought, "this isn't what I'm supposed to be doing," even as greater forces pull me forward. This is known as the refusal of the call to adventure. It's a routine part of adventure.

3.7 There is a criticism you could make of my advocacy of surrender. It is: "you're just lucky and narrativizing it." Yes. If you, too, have luck that seems unrelated to your conscious choices, consider letting that run the show.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. [Vaults](#) in [Claude Managed Agents](#) covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

Making MCP clients more context-efficient

MCP standardizes how AI agents ([clients](#)) connect to and work with tools and data sources they need ([servers](#)). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

Load tool definitions on demand with tool search

[Tool search](#) defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our [testing](#), tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: [advanced tool use](#)

Process tool results in code with programmatic tool calling

[Programmatic tool calling](#) processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly [37%](#) on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See [advanced tool use](#) for the full breakdown.

Pairing MCP servers with skills

Skills and MCP are complementary. MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

Bundle skills and MCP servers as a plugin

Plugins for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our data plugin for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: Extending Claude's capabilities with skills and MCP servers

Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. Canva, Notion, Sentry, and more do this today in Claude, listing the skill next to their connector in our web directory.

To make that pairing portable across every client, the MCP community is actively working on an extension for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

1.5 I've discovered that scrutinizing my motives too closely is a great way to stop creating. The unexamined life may not be worth living, but the fully examined life is a pinned butterfly in a glass case. I've concluded that my life is a grand artistic project, but I'm best off not knowing what it is.

1.6 You know that person who had important ripple effects in your life, based on a brief meeting or a kind gesture they extended to you, which meant nothing to them, but everything to you? Or that interesting stranger whose manner cast a brief spell on you, lingering in your memory disproportionately? That's you also, you're that person. Most of your impact on others, you will never fully apprehend or appreciate. Thus, it's not important to know what you're doing.

1.7 The proper end of introspection is not being filled with facts about oneself. Your introspection is successful if it is finite, if it leads you towards direct contact with experience. Get out of the way of your becoming.

Choice and authenticity

2.1 People feel most themselves when they are unselfconscious, which is to say, their actions emerge without apparent control. Think of the phrase "singing with abandon." In our most profound moments of intimacy, we have abandoned the wheel. Who is driving, at that moment? What if we simply let that driver have the wheel more often? It takes practice to notice when that's the person taking over the wheel—as opposed to our reactive, fear-based conditioning, which is also eager to take the job. Surrender is a skill.

2.2 When people talk about "being authentic," they are often referring to executing some program, some persona, that they've devised privately. Of course, our impression of such people is that they're unnatural and unoriginal. They assign us the job of reinforcing a character we never agreed on.

2.3 Surrender is something that happens through you, not something chosen^[2]. I couldn't have skipped the part of my life when I was contriving everything. And the way I'm living now wouldn't work without the skills I accumulated during that period of contrivance. If you're reading this and thinking, "gosh, I wish person x would see the wisdom of surrender," you have discovered an opportunity to surrender to interpersonal helplessness.

2.4 Often, the apparent struggle to make a decision is an attempt to meet an internal flailing quota, after which we will feel affirmed in the choice we already knew we were making. Sometimes you can just skip all of that, and admit that you're scared of what you know you will do. And then do it.

Some years after making that decision, I'm much happier. My life is richer, and I'm doing more for others. Regularly, I react with positive disbelief when I look at what my life has become. It looks like I've been executing a genius strategy. But in reality, I am living in a state of ongoing confusion, just seeing what happens.

If you've achieved a bunch of your goals, and you're still not happy with your life, consider giving up on self-chosen plans. Instead, listen to your life, inner and outer, and see if you can feel a current. Then fall into it, especially if it has nothing to do with the story you had about where you were supposed to be going.

This choice is known as surrender. As with most cliches, I don't think I can explain the profound experiential content of surrender—what it feels like, why it works. But perhaps I can lay out a constellation of propositions^[1] that will, together, illuminate a worldview very different from the one I once inhabited.

Knowing and not-knowing yourself

1.1 The planning part of your mind, the one that generates ambitions and hypothetical situations, is a small part of your overall intelligence. Anyone who is halfway good at sex, comedy, or any creative art will know this. Designing your life based on what that part of your mind wants is like designing your house to fit the needs of your right hand. Congratulations, your house is filled with fidget spinners and soap.

1.2 A large percentage of human suffering is anticipatory tension, or dreading a future experience such that you actualize its potential suffering in the now. Anticipatory tension is using the illusion of knowledge to generate the illusion of control. Much of what people call planning is this.

1.3 My best outfits are all selected by someone else. When I'm at a clothing store, and I meet an employee who seems engaged, I tell them to dress me for maximum attractiveness. They do a much better job than I do. Of course: I can't see myself as well as someone else can. This is why I try to listen to what the world apparently wants to do with me, rather than who I imagine I am.

1.4 Often, our self-image is a defense against the traits we reject in ourselves. "I love to be spontaneous," says the most calculating person you've ever met. Thus, if you're trying to live in self-loathing and denial, one way is to sculpt your life around the self-image you've decided on.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

Headless everything for personal AI

INTERCONNECTED · 18 APR 2026 · [SOURCE](#)

It's pretty clear that apps and services are all going to have to go *headless*: that is, they will have to provide access and tools for personal AI agents without any of the visual UI that us humans use today.

By services I mean things like: getting a new passport; finding and booking a hotel or a flight; managing your bank account; shopping for t-shirts with a minimum cotton weight from brands similar to ones that you've bought from before.

Why? Because using personal AIs is a better experience for users than using services directly (honestly); and headless services are quicker and more dependable for the personal AIs than having them click round a GUI with a bot-controlled mouse.

Where this leaves design for the services... well, I have some thoughts on that.

Headless services? They're happening already.

Already there is [MCP, as previously discussed](#) (2025), a kind of web API specifically for AI. For instance, best-in-class call transcriber app Granola [recently released their MCP](#) and now you can ask your Claude to pull out the meeting actions and go trawling in your personal docs to find answers to all the question. A good integration.

But command-line tools are growing in popularity, although they used to be just for developers. So you can now create a spreadsheet by typing in your terminal:

```
gws sheets spreadsheets create --json '{"properties": {"title": "Q1 Budget"}}'
```

Here are some recently launched CLI tools:

- [gws](#): Google Workspace CLI, Drive, Gmail, Calendar, and every Workspace API. Zero boilerplate. Structured JSON output. 40+ agent skills included.
- [Obsidian CLI](#) to do anything you can do with the (very popular) note-taking app - like keep daily notes, track and cross off tasks, search - from the CLI
- [Salesforce CLI](#) and, look, I don't really understand what the Salesforce business operating system does, but the fact it has a CLI too is significant.

And here is [CLI-Anything](#) (31k stars on GitHub) which auto-generates CLIs for ANY codebase.

Why CLIs?

It turns out that the best place for personal AIs to run is on a computer. Maybe a virtual computer in the cloud, but ideally your computer. That way they can see the docs that you can see, and use the tools that you can use, and so what they want is not APIs (which connect webservers) but little apps they can use directly. CLI tools are the perfect little apps.

CLIs are composable so they are a better fit for what users actually do.

By composable I mean you can: query your notes then jump to a spreadsheet then research the web then jump back to the spreadsheet then text the user a clarifying question then double-check your notes, all by bouncing between CLIs in the same session.

A while back app design got obsessed with “user journeys.” Like the journey of a user finding a hotel then booking a hotel then staying in it and leaving a review.

In the cloud era, enterprises accumulated data. In the AI era, they accumulate learning. The trust boundary must evolve accordingly, from protecting information to protecting the mechanisms through which organizations learn, adapt, and compound intelligence. There are a few things every enterprise must do to ensure this:

Control: Create your private evals, because evals define what “good” looks like inside the organization. Also, retain ownership of your organization’s memory, traces, feedbacks, decisions, and institutional context, and ability to use outputs of models from your own tasks and queries.

Capability: Build your own proprietary learning environments within the tenant boundary to train or tune models, where models learn against real workflows without exposing the company’s knowledge.

Choice: Ensure the orchestration layer is decoupled from any single model. Ask yourself: If any one model you are using is taken away, do you still have the ability to operate and optimize for your evals using other models? Does your company “veteran” capability remain with you even if a given “generalist” model is taken away?

Cost: By decoupling the orchestration layer, you are also able to bring together context, models, and tasks in the most efficient and cost-effective way without sacrificing quality.

Compound: Bring these four together and you create your own continuous learning loop (i.e. hill climbing machine) that will allow your AI investments to compound the value of your firm.

In other words, a company should be able to use a model without giving up the knowledge that makes it unique. That is the reverse information paradox we need to confront.

Surrender as a non-stupid life strategy

SASHA CHAPIN · 04 JUL 2026 · [SOURCE](#)

After getting everything I wanted in my 20s, and discovering that I was still miserable, I decided that I should no longer be in charge of choosing what I want, and I slowly settled into a frame of, “I’ll do whatever God seems to want from me.”

Over time, the information asymmetry becomes increasingly skewed. The seller learns more and more about you as you use what you purchased, while you learn very little about what the seller is learning in return.

That is what I think of as the Reverse Information Paradox.

Patents solve one aspect of Arrow's paradox. They let an inventor disclose an idea without simply giving it away. The Reverse Information Paradox needs its own equivalent.

This requires more than data protection. Models learn from "exhaust," the prompts people write, the tools agents use, and especially the corrections people make when the model is wrong. Every correction is distilled into institutional know-how. It's the kind of knowledge a competitor could never buy, and the kind that leaks almost imperceptibly: trace by trace, correction by correction, eval by eval.

In consuming intelligence, you are creating intelligence. And what you create should belong to you. This is your particular intelligence, in Hayek's sense: the knowledge of time, place, and circumstance that no one else can hold. It knows what you think, what you value, and how you measure success.

While the great innovation that comes from model providers having fair use rights to train models on public data is needed, I find it ironic that the status quo is to then turn around and impose restrictive terms on distillation, and to reserve the right to learn from customer usage and interaction data. If learning flows in only one direction, economic value converges toward the owners of the learning infrastructure rather than the creators of the knowledge itself. Therefore, it's imperative that we distribute the learning infrastructure to every firm so that they can control their own learning loop.

As Alex Karp put it: "What the technical customers want is control over their compute, their models, their data stack, and their alpha. They want to know they own the means of production, and it's not being transferred to someone else." The current regime does precisely the transfer Karp and companies fear.

That is why enterprises need a real trust boundary for their human capital and token capital to compound. It is where an organization's data, traces, evals, adapted weights, and memory accumulate and improve together. And it is a hard boundary across which nothing crosses, not even the intelligence exhaust, without consent. Enterprises will demand the rights to use model outputs to fine tune and/or train their own models. I think of this as every firm's right to align models to their enterprise accountability obligations.

But users don't live in "journeys." They multitask; they talk to people and come back to things; they're idiosyncratic. Try grabbing the search results from the Airbnb app and popping them in a message to chat on the family WhatsApp, then coming back to it two days later. It's a pain and you have to use screenshots because apps and their user journeys are not composable.

CLIs are composable because they came originally from Unix and that is the Unix tools philosophy: tools were designed so that they could operate together.

Personal AIs like Openclaw or [Poke](<https://poke.com>) do what users want and don't follow "designed" user journeys, and as a result the composed experience is more personal and way better. CLIs are a great underlying technology for that.

CLIs are smaller than regular apps and so they are easier to secure.

The alternative to providing special tools for AIs is that the AIs use the same browser-based apps that we do, and that's a terrifying prospect.

For one, AIs are really good at finding security holes. The new Mythos model from Anthropic is so good at discovering security flaws that it has been held back from the public and governments are convening emergency meetings of the biggest banks.

And including a UI increasing complexity and makes security holes more likely.

Here's a recent shocking example. Companies House is the national register of all companies, directors, and accounts for England and Wales. Users could view *and edit* any other user's account:

a logged-in company director could exploit the flaw by starting from their own dashboard and then trying to log into another company's account.

Once they reach the 2FA block, which they would not be able to pass, all that was required was to click the browser's back button a few times. Typically, the user would be taken back to their own dashboard, but the bug instead returned them to the company they had tried to log into but couldn't.

This bug had been present since October 2025.

Imagine a future where personal AIs are filing company records, and one of them notices this security hole overnight and posts about it on moltbook or whatever agent-only social network is most popular. The other agents would exploit the system up, down and sideways before the engineering team woke up.

The only viable solution is that services need to security hardened, and to do that they need to be simplifying and minimised. Again, CLI tools are a great fit.

What does this mean for front-end design?

Design won't go anywhere.

Sure, the front-end should be driving the same CLI tools that agents use.

Arguably it's more important than ever: human users will encounter services, figure out what they can do, and pick up their vibe from using the app, just as now.

Then they'll tell their personal AI about the service and never see the front-end again, or re-mix it into bespoke personal software.

So from a usability perspective I see front-end as somewhat sacrificial. AI agents will drive straight through it; users will encounter it only once or twice; it will be customised or personalised; all that work on optimising user journeys doesn't matter any more.

But from a vibe perspective, services are not fungible. e.g. if you're finding a restaurant then Yelp, Google Maps, Resy, and The Infatuation are all more-or-less equivalent for answering that question but clearly they are completely different and you'll use different services at different times.

Understanding that a service is *for you* is 50% an unconscious process - we call it brand - and I look forward to front-end design for apps and services optimising for brand rather than ease of use.

If I were a bank, I would be releasing a hardened CLI tool like *yesterday*.

There is so much to figure out:

- How do permissions work? Should the user get a notification from their phone app when the agent strays outside normal behaviour? How do I give it credentials to act on my behalf, and how long do they last?
- How does adjacency work? My bank gives me a current account in exchange for putting a "hey, get a loan!" button on the app home screen. How do you make offers to an agent?

Headless banks.

Headless government?

I'd love to show you a worked example here. I vibed up a suite of four CLI tools that wrap four different services from UK government departments.

If I were renting a house, I would set my agent to learning about the neighbourhoods using one of these tools. Another tool will be helpful next time I'm buying a used car. (There's a Companies House command-line tool too.)

But I won't show you the tools because I don't want to be responsible for maintaining and supporting them.

I wish Monzo came with an official CLI. I wish Booking.com came with a CLI. I reckon, give it a year, they will.

Auto-calculated kinda related posts:

If you enjoyed this post, please consider sharing it by email or on social media. [Here's the link](#). Thanks, —Matt.

The Reverse Information Paradox

SN SCRATCHPAD · 05 AUG 2026 · [SOURCE](#)

Notes on advances in technology and real-world impact

In the age of intelligence, how should firms protect their core IP?

Nobel Prize winning economist Kenneth Arrow [famously described](#) a paradox in the market for information. "Its value for the purchaser is not known until he has the information, but then he has in effect acquired it without cost." In Arrow's "Information Paradox," the seller risks giving away knowledge in order to sell it.

AI creates the reverse problem. In the AI age, the buyer risks giving away knowledge, just in order to use what they bought.

You essentially pay for intelligence twice, once with money, and again with something even more valuable: the proprietary knowledge you must reveal to make that intelligence useful. The better you want the model to perform, the more of that knowledge you have to feed it!